> **Code, raw capture and all extracted data:**
> https://github.com/Vatsalya001/beyond_labs_assignment

ChatGPT Retrieval Leak — capturing and parsing the network stream

Scope: ChatGPT only, free account, **one prompt, one conversation.** No other platforms. **Time spent:** ~90 minutes end to end. **Tools:** Chrome DevTools, a hand-written `fetch` interceptor, Python 3 (stdlib only), `jq`, Claude Code (Opus) for scaffolding scripts in parallel with the browser work.

Prompt used:

> best project management tools for small agencies in 2026 — include what recent YouTube reviews say

Captured: `chatgpt.com/backend-api/f/conversation`, 377,242 bytes, 19.89 s stream, model `gpt-6`.

---

1. Headline findings

1. Retrieved ≠ Cited — confirmed. 25 sources retrieved → 7 cited in the final answer = 28% conversion. 72% of what ChatGPT pulled never reached the answer.

2. One prompt became 2 silent sub-searches. Neither is visible anywhere in the UI:

1. `best project management tools for small agencies 2026 comparison pricing features Asana ClickUp Monday Teamwork Basecamp`
2. `YouTube project management software review 2026 small agency ClickUp Asana Teamwork Monday.com`

3. The YouTube "rich metadata" claim did NOT reproduce in my capture. This is my most interesting result, and it's a negative one. The referenced research reports that retrieved YouTube videos carry title, channel name, follower count, verified status, publish date, views, likes and the full description. In my stream, YouTube entries carried:

```
{
  "type": "search_result",
  "url": "https://www.youtube.com/watch?v=SzYhFDX_TV0",
  "title": "ClickUp vs Monday.com vs Asana - Which is Best? (2026) - YouTube",
  "snippet": "",
  "ref_id": { "turn_index": 708623, "ref_type": "youtube", "ref_index": 5 },
  "pub_date": 1760572800.0,
  "attribution": "www.youtube.com"
}
```

Empty snippet. No channel, no followers, no verified flag, no views, no likes, no description. I audited all 14 claimed field names against the entries *and* against the raw 377 KB stream — every one absent (full table in `retrieved_vs_cited.md`).

So the "no transcript" part is trivially true here, but for a stronger reason than the original claim: there was no video text content of any kind. My hypothesis is in §5.

4. ChatGPT has a distinct `reddit` retrieval channel. `ref_type` values observed: `search` (21), `youtube` (2), `reddit` (2). Video and Reddit sources are typed separately from ordinary web results, which implies separate retrieval/scoring paths. Both Reddit threads were retrieved and neither was cited.

5. The response stream contains advertising. `ads_response_id`, `single_advertiser_ad_unit`, and a sponsored card (`"title": "Tom Frazier", "subtitle": "Sponsored"`) are interleaved with the answer content on the free tier.

---

2. Outputs

File	What it is
`out/retrieved_vs_cited.md`	The headline stats, fan-out queries, YouTube field audit, full 25-source table
`out/citations.json` / `.csv`	**Task 1** — every retrieved source: url, title, snippet, pub_date, domain, ref_id, cited flag, cited_via
`out/fanout_queries.json`	**Task 2** — the sub-queries + citation-join diagnostics
`out/youtube_snapshot.json`	The YouTube entries + the 14-field audit
`out/key_inventory.txt`	The discovery pass over the raw payload (what keys actually exist)
`out/answer.txt`	Reassembled answer text, 9,502 chars
`capture/chatgpt_raw.txt`	The raw captured stream — immutable source of truth

Code: `capture/interceptor.js`, `parse/inventory.py`, `parse/parse_chatgpt.py`

---

3. How I captured it

Forced the search. On free tier the router may answer from weights, which yields a stream with no search activity at all. I enabled the composer's Web search tool explicitly rather than hoping — a no-search stream would have wasted the capture window.

Tapped `fetch` instead of copy-pasting from DevTools. Pasted an interceptor into the console *before* sending:

```
const [toPage, toUs] = res.body.tee();   // page renders from one branch, we drain the other
// ...
store[key] += dec.decode(value, { stream: true });  // streaming decode
```

Two details that matter:

- `res.body.tee()` splits the `ReadableStream` into independent branches, so nothing is consumed twice and the UI is unaffected.
- `TextDecoder` in streaming mode — a naive per-chunk `decode()` corrupts any multi-byte character split across a chunk boundary.
- It must be installed **before** the request fires. A page-load-time request would need an extension injecting at `document_start`.

Result matched byte-for-byte: console reported `377,242 chars`, file on disk is 377,242 bytes.

What I learned about capture methods: HAR export does **not** preserve streamed/SSE response bodies — you get the request with empty content. The DevTools EventStream tab works as a manual fallback. I kept both paths available.

---

4. How I parsed it — three traps

Trap 1: the stream is not uniform JSON

107 frames, **50% of them JSON-patch deltas** like:

```
{"c": 12, "o": "append", "p": "/message/content/parts/0", "v": "text fragment"}
```

These mutate a message announced in an *earlier* frame. A line-by-line `json.loads` parser reads each frame in isolation and silently drops everything that only exists as a patch. Reassembly must be **stateful**: track the currently-streaming message and apply JSON pointers to it.

Trap 2: every delta is sent twice

Each content delta appears once live and once through a revision / `streaming_parent_patch` mirror. Without dedupe the reassembled answer is **literally doubled**. My parser skipped 14 mirrored duplicates.

Trap 3: the citation scheme was not what I expected

I went in expecting inline `citeturn0search3` markers. That string appears **zero times** in the stream. What's actually there:

- **Inline in the answer text**, as JSX-like markup: `<Cite refs={["turn828649search4","turn828649search8","turn828649search9"]}/>`
- **As `type: "cite"` objects** carrying `safe_urls[]` and `entities[].ref`
- Those refs resolve against each retrieved source's `ref_id: {turn_index, ref_type, ref_index}`

Note the turn id is `828649`, not `0` — so even a regex written for the right *shape* would have failed if it hardcoded `turn0`.

This is why the extractor is name-agnostic. It finds source records by *shape* (any object carrying a URL-ish value plus a title-ish value) and derives the specifics from a discovery pass (`inventory.py`) over the actual payload, rather than trusting any documented field list. These are undocumented private APIs; the names drift.

Cross-validation

I resolved "cited" two independent ways: (a) URLs listed in `safe_urls` of `cite` objects, (b) `entities[].ref` resolved back through `ref_id.ref_index`. Both agree — **0 unresolved refs, 0 cited URLs outside the retrieval set.** Six sources matched on both signals; the YouTube video matched on `safe_url` only.

---

5. What did not work

What I tried	What happened	Why	What I did instead	With more time
Assumed `citeturn0search3` inline markers	Zero matches in 377 KB	That scheme isn't used here; it's `<Cite refs={[...]}/>` + `type:cite` objects, and the turn id is `828649` not `0`	Ran a discovery pass first, then matched on structure	Build a scheme-detector that tries several known citation formats and reports which matched
Counted unique URLs naively	Got **45** retrieved	The stream carries each page twice — clean and `?utm_source=chatgpt.com`	Canonicalised URLs by stripping tracking params → **25**	Canonicalise host + trailing slash + AMP variants too
Joined citations on `(turn_index, ref_type, ref_index)`	Half the joins failed silently	The same source appears with **two different `turn_index` values** (`0` in the UI copy, the real turn id in the tool copy); my dedupe kept the wrong variant	Indexed refs from *all* raw entries, with a loose `(ref_type, ref_index)` fallback	Keep every variant and reconcile explicitly instead of picking a winner
Grepped for transcript-shaped keys	Hit `"subtitle"` — looked like a caption track	False positive: it was an **ad card's** `{"title":"Tom Frazier","subtitle":"Sponsored"}`	Tightened the pattern and re-checked with surrounding context	Always print match context before asserting a negative finding
Looked for the rich YouTube text blob	Not present	The `web.run` tool messages arrived with **empty content** — the browser got the UI-rendering copy of retrieval, not the raw tool text the model consumes	Audited all 14 claimed fields and reported the absence as the finding	See hypothesis below
Expected reasoning content	`thoughts` and `reasoning_recap` messages exist as *types* but their content never streamed	Likely withheld on free tier / this model routing	Noted it	Re-test on a paid account and a reasoning model

Hypothesis for the YouTube non-reproduction

The browser receives a **UI-rendering projection** of retrieval (`search_result_groups`: url, title, snippet, pub_date, attribution, ref_id) rather than the raw tool output the model reads. In my capture the `web.run` tool messages were streamed as **empty shells**. The richer YouTube payload the original research describes is most likely inside that tool output, which on this account/model was never sent to the client.

Supporting evidence (desk research, not my capture): the YouTube stats fields appear in **no** published type definition of `search_result` metadata across the maintained open-source ChatGPT schema/exporter projects. That is consistent with them not being structured JSON fields at all, but a **pre-rendered text blob** inside the `web.run` tool result — the format in which each retrieved source is prefixed with its `【turn{N}{type}{M}】` token and a `[wordlim N]` budget label. If that is right, the original finding was read out of the *tool-result text*, and reproducing it requires a capture in which that text actually streams. Mine did not.

This also explains the shape of what I *did* get: `ref_type: "youtube"` exists as a first-class source class (alongside `search` and `reddit`), so ChatGPT clearly treats video as a distinct retrieval channel — I just never

received its payload.

How I'd test it, cheapest first:

1. `GET /backend-api/conversation/{conversation_id}` returns the whole turn as plain JSON with metadata already reconstructed — no SSE parsing at all. Perfect ground truth to diff against my stream parser, and it would immediately show whether the tool-result text is retained server-side.
2. Same prompt on a Plus/Pro account and on a reasoning-enabled model.
3. A query where the video *is* the answer, so the model is forced to read it.

Diff `ref_type: youtube` entries across all three. If the rich fields appear on a paid tier, the correct claim becomes **"the leak is tier-dependent"** — more precise, and more useful, than the original.

---

6. Main task — other platforms: method, not findings

I did not capture other platforms. I had a free ChatGPT account and no logged-in sessions elsewhere, so I chose depth on one platform over breadth across four. Rather than assert things I haven't verified, here is the method, which transfers unchanged:

1. DevTools → Network → find the request that stays open for the duration of the answer. Note its `content-type`: `text/event-stream`, `application/x-ndjson`, chunked JSON, or a WebSocket.
2. Install the same `fetch` tee interceptor (plus XHR / EventSource / WebSocket shims if the transport isn't `fetch` — my interceptor already includes the first two).
3. Run `inventory.py` on the capture. It assumes **nothing** about field names, so it works on any payload: frame shapes, type discriminators, biggest arrays, every URL with its key path, every long string. The retrieval set is almost always "the one big array of objects each holding a URL, a title and a long snippet."
4. Run the shape-based extractor. Only the citation-join logic and the patch-frame key names are ever platform-specific.

The axes I'd compare on, and why each matters:

Axis	Why it matters
Transport	Decides whether a `fetch` tee is sufficient
Cumulative snapshots vs incremental deltas	Decides whether a reassembly pass is needed — the main trap on ChatGPT
Keyed objects vs positional arrays	Positional RPC envelopes force purely shape-based extraction
Fan-out visibility	Are sub-queries streamed, or resolved server-side and never sent?
Retrieved-set visibility	Does the client get the full retrieval universe or only what survived into the answer? A platform that ships only the cited set **cannot** answer the retrieved-vs-cited question at all
Per-source metadata richness	Snippet, date, and platform-specific extras
Opaque / encrypted result fields	If results arrive encrypted, snippet extraction is blocked — itself a finding

Cheapest next platform: Perplexity — it answers without a login, so it costs one capture and no account.

Is it automatable? Yes, and it already is: `parse_chatgpt.py` is the repeatable script and `inventory.py` is the tool that bootstraps a new platform adapter in minutes.

---

7. Scaling this to a system

The hard problem is not capture. It is silent schema drift. These are undocumented private APIs that change with no notice, no versioning and no deprecation window. A pipeline built on hardcoded field paths doesn't crash when they change — it quietly returns zero citations, and your dataset develops a hole you notice weeks later. I hit a miniature version of this three times in 90 minutes (the citation scheme, the utm duplicates, the dual `turn_index`).

```
Capture      Headful Chrome, persistent authenticated profiles (one per platform
             × account), driven over CDP. Interceptor injected at document_start
             in the MAIN world. Residential proxy pool — search results are
             localised, so geo is an experimental variable, not an afterthought.
             CAPTCHA detection → quarantine the profile and alert; never retry
             blindly into a ban.
   ↓ raw bytes, nothing parsed
Raw store    Every stream → object storage, compressed, keyed run_id/platform/date.
(IMMUTABLE)  Never mutated. WHY: when an adapter turns out to be wrong — and mine
             was, twice — you reprocess history instead of re-collecting it.
             Re-collection is impossible: the model, the index and the SERP have
             all moved on.
   ↓
Normalise    Versioned per-platform adapters behind one interface. The shape-based
             extractor stays available as a permanent fallback.
   ↓
Analytics    Postgres/DuckDB: runs, fanout_queries, retrieved_sources,
             cited_sources, source_attributes.
   ↓
Research     • Citation share by domain over time
             • Retrieved→cited conversion per domain (being retrieved is cheap;
               being cited is the prize — that 28% is the real ranking signal)
             • Which attributes correlate with being cited
             • Fan-out patterns: how one prompt becomes K searches
```

Unified schema — one row per run, with `raw_attributes` preserving everything the platform sent (so a field you didn't know mattered is still recoverable) and `raw_blob_ref` so any normalised row can be regenerated from source:

```
{
  "run_id": "2026-10-09T19:42Z/chatgpt/a1b2c3",
  "platform": "chatgpt", "model": "gpt-6", "account_tier": "free", "geo": "IN",
  "prompt": "...",
  "fanout_queries": [{"order": 1, "query": "...", "tool": "web.run"}],
  "retrieved": [{
    "rank": 5, "url": "...", "canonical_url": "...", "domain": "youtube.com",
    "title": "...", "snippet": "", "publish_date": "2025-10-16",
    "source_type": "youtube",
    "ref_id": {"turn_index": 708623, "ref_type": "youtube", "ref_index": 5},
    "raw_attributes": { "...everything the platform sent..." }
  }],
  "cited": [{"canonical_url": "...", "via": ["safe_url", "ref_id"],
             "refs": ["turn828649search4"]}],
  "answer_text": "...", "reasoning_text": null,
  "raw_blob_ref": "s3://.../a1b2c3.ndjson.gz",
  "adapter_version": "chatgpt@2026-10-09",
  "extraction_warnings": []
}
```

Drift detection — the part most people skip:

1. **Canary assertions per run** — ≥1 fan-out query, ≥5 retrieved, ≥1 citation, non-empty answer. Violation → alert, quarantine, don't write to analytics.
2. **"Zero citations extracted" alarm** — the single highest-value monitor in the system.
3. **Key-inventory diff** — store observed JSON key paths per run, diff against the previous run for that platform, alert on newly-appeared or disappeared keys. Catches drift *before* it breaks extraction. This is just `inventory.py` run on a schedule.
4. **Payload shape hash** over the frame-shape distribution — a step change means the platform shipped something.
5. **Automatic fallback** — when a named-path adapter misses its canary, re-run through the shape matcher and flag the row `extraction_warnings: ["fell_back_to_shape_matcher"]`. Degraded data, clearly labelled, beats silent zeros.

n8n variant: workable as the orchestration layer — Cron → queue → container step that drives the browser → Code node for normalisation → Postgres insert → Slack alert on canary failure. Wrong tool for the capture itself: browser-profile lifecycle, CDP streaming, CAPTCHA quarantine and proxy rotation are stateful long-running concerns that want a real service, not a workflow node.

On ToS and durability — what I'd tell a client: this reads traffic my own browser already receives on my own account, which is fine for research at small scale. Automating logged-in consumer accounts at volume sits against most providers' terms, and the transport is undocumented and unversioned. Treat the pipeline as a **research instrument with a maintenance budget**, not as infrastructure. Keep the raw blobs so history stays reprocessable, respect rate limits, prefer official APIs where they expose the same data, and keep capture volume proportionate to the question being asked.

---

8. What I'd do next, in priority order

1. **Resolve the YouTube question** — same prompt on a paid account and a reasoning model, diff the `ref_type: youtube` entries. Determines whether the rich metadata is tier-dependent or gone.

2. **Scale to N queries on one platform** before adding platforms — retrieved→cited conversion is only meaningful across a corpus. One query gives 28%; it takes ~100 to know whether that's typical.
3. **Perplexity** as platform #2 (no login required), then Gemini.
4. **Ship `inventory.py` as the drift monitor** — it's already the right tool, it just needs a scheduler and a diff.
5. Investigate the `reddit` ref_type — a separate retrieval channel for forum content is a notable signal if it holds across queries.